

بازیابی اطلاعات

دانشگاه آزاد شیراز - دانشکده مهندسی کامپیوتر

ترم دوم سال تحصیلی ۱۴۰۴-۱۴۰۵

فصل اول: Boolean retrieval

استاد: دکتر امین اسکندری

تیم طراحی محتوای آموزشی و ویراستاری (علمی و ادبی): حمید نامجو، امیرحسین همتی و علی مجاهد

فهرست مطالب

0 Introduction to IR

1.1 An example information retrieval problem

1.2 A first take at building an inverted index

1.3 Processing Boolean queries

1.4 The extended Boolean model versus ranked retrieval

1.5 Chapter 1 Summary

۱. مقدمه‌ای بر بازیابی اطلاعات: سفر به قلب وب امروزی

تصور کنید یک مغز دیجیتال فوق‌العاده

شما هر روز چند بار از خود می‌پرسید: «اون ایمیل مهم کجا رفت؟»، «مقاله‌ای که دیروز خوندم رو کجا ذخیره کردم؟» یا «چطور می‌تونم یه فیلم خوب با بازیگرای مورد علاقه‌م پیدا کنم؟». در پشت هر یک از این سوالات ساده، یک گول نامرئی به نام بازیابی اطلاعات (Information Retrieval) کار می‌کند. این علم، هنر یافتن سوزن اطلاعات در انبار کاه دیجیتال ماست.

دهه‌ها پیش، این یک مهارت تخصصی برای کتابداران و وکلای پژوهشگر بود. امروز، اما صدها میلیون نفر هر روز از آن استفاده می‌کنند—هر بار که در گوگل عبارتی را تایپ می‌کنند یا در صندوق ورودی ایمیل خود کلمه‌ای را جستجو می‌نمایند. بازیابی اطلاعات، دیگر یک فعالیت حاشیه‌ای نیست؛ بلکه به dominant form of information access تبدیل شده است.

چالش اصلی: داده‌های بی‌نظم و نامنظم

قلب این علم، کار با داده‌های ناساختاریافته (Unstructured Data) است. داده‌های ساختاریافته، مانند یک جدول اکسل منظم یا یک پایگاه داده فروشگاه، ستون‌ها و ردیف‌های مشخصی دارند. اما بیشتر اطلاعات دنیای ما—مانند ایمیل‌ها، مقالات،

اخبار و صفحات وب—مثل یک اتاق پر از وسایل هستند که هرچیز جایی ریخته و هیچ برچسبی ندارد. علم بازیابی اطلاعات به کامپیوتر یاد می‌دهد که چطور در این اتاق شلوغ، مورد مورد نظر ما را پیدا کند.

البته، تقریباً هیچ داده‌ای کاملاً بی‌ساختار نیست. متون هم دارای ساختارهای پنهان زبانی هستند. سیستم‌های بازیابی مدرن از این ساختارها مانند عناوین، پاراگراف‌ها و حتی لینک‌ها (hyperlinks) بهره می‌برند تا جستجو را **نیمه‌ساختاریافته** (Semi-structured) و هوشمندانه‌تر کنند.

📁 فراتر از جستجو: سازماندهی و طبقه‌بندی

بازیابی اطلاعات فقط پیدا کردن چند سند مرتبط نیست. این شامل وظایف هوشمندانه‌تری است:

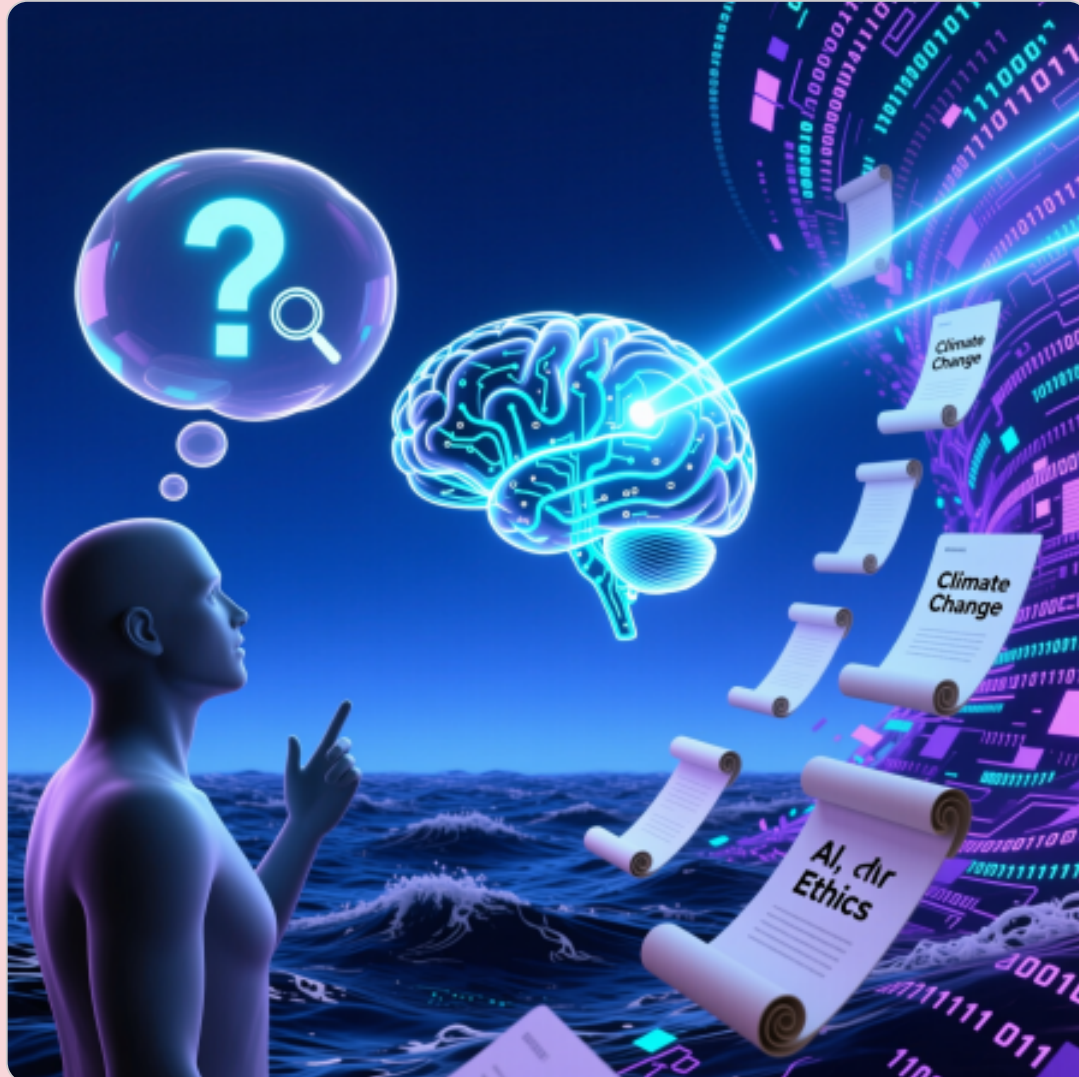
- **خوشه‌بندی (Clustering):** تصور کنید صدها سند علمی در اختیار دارید. سیستم می‌تواند آن‌ها را به صورت خودکار به گروه‌های موضوعی تقسیم کند، درست مانند زمانی که کتاب‌های یک قفسه را بر اساس موضوع مرتب می‌کنید.
- **طبقه‌بندی (Classification):** سیستم ایمیل شما هر روز این کار را برای شما انجام می‌دهد. به صورت خودکار تشخیص می‌دهد که کدام ایمیل اسپم است و کدامیک مهم و آن را در پوشه مناسب قرار می‌دهد.

با توجه به مقیاس عملکرد، سیستم‌های بازیابی اطلاعات به سه دسته اصلی تقسیم می‌شوند:

1. **جستجوی وب (Web Scale):** بزرگترین و پیچیده‌ترین حوزه. این سیستم‌ها باید بر روی میلیاردها سند در میلیون‌ها کامپیوتر جستجو کنند. چالش‌ها عظیم هستند: از جمع‌آوری اسناد با خزنده‌های وب (Crawlers) گرفته تا مقابله با ترفندهای سایتی برای دستکاری رتبه‌ها.
2. **جستجوی شخصی (Personal Search):** همان جستجویی که در کامپیوتر خود انجام می‌دهید (مانند Spotlight در مک یا Instant Search در ویندوز). اینجا تمرکز بر سرعت، سبکی و کارایی با منابع محدود است.
3. **جستجوی سازمانی/حوزه‌محور (Enterprise/Domain-Specific):** جستجو در مجموعه‌های متمرکز مانند اسناد داخلی یک شرکت، پایگاه‌های ثبت اختراع یا مقالات تخصصی شیمی. این سیستم‌ها معمولاً روی چند سرور قدرتمند اجرا می‌شوند.

🖼️ یک تصویر به ارزش هزاران کلمه

برای درک عمیق‌تر این مفهوم، تصور کنید کاربری که در اقیانوس بی‌پایان داده‌ها (سمت راست) یک نیاز اطلاعاتی دارد (سمت چپ). یک سیستم بازیابی اطلاعات هوشمند (مغز دیجیتال) باید بتواند از میان این حجم عظیم داده‌های نامنظم، دقیقاً همان اسناد مرتبط را استخراج و به کاربر ارائه دهد. این تصویر، قلب تلاش ما در این فصل است: چطور این مغز دیجیتال را بسازیم.



در این کتاب، ما به بررسی ایده‌ها و تکنیک‌هایی می‌پردازیم که این مغز دیجیتال را می‌سازند؛ از ساختارهای داده‌ای اولیه گرفته تا الگوریتم‌های پیچیده‌ای که دنیای وب را می‌چرخانند.

۱.۱ یک مسئله نمونه از بازیابی اطلاعات: جستجو در اقیانوس دیجیتال 🐟

🗨️ تصور کنید مدیر یک سرویس استریم فیلمنامه

شما مدیر آرشیو دیجیتال غول‌پیکری هستید که شامل تمام فیلمنامه‌های نوشته‌شده در تاریخ است. یک روز، یک کاربر پژوهشگر با یک درخواست بسیار خاص به شما مراجعه می‌کند: «می‌خواهم تمام فیلمنامه‌هایی را پیدا کنم که در آن‌ها **بروتوس** و **سزار** هر دو حضور دارند، اما **کالپورنیا** در هیچ‌کدام حضور ندارد.» چطور این جستجو را به صورت کارآمد انجام می‌دهید؟

یک راه اولیه و شاید بلافاصله به ذهن می‌رسد، استخدام یک کتابدار بسیار پرتلاش و خواندن تک‌تک تمام فیلمنامه‌ها! این روش در دنیای کامپیوتر به **پیمایش خطی** یا **گِرپ کردن (grepping)** معروف است. برای مجموعه‌ای به کوچکی مانند آثار کامل شکسپیر، این روش حتی با سرعت کامپیوترهای امروزی هم قابل قبول است. اما برای میلیاردها سند یا برای پرسمان‌های پیچیده‌تر، این روش مانند تلاش برای پیدا کردن سوزن در انبار کاه است!

🧩 ایده اول: ماتریس وقوع بولی (یک برگه بازرسی غول‌پیکر)

اولین راهکار هوشمندانه، ساخت یک **ماتریس وقوع بولی** است. تصور کنید یک برگه غول‌پیکر که در آن هر ردیف به یک واژه (مثل «بروتوس») و هر ستون به یک سند (مثل «هملت») اختصاص دارد. در خانه‌ای که واژه در سند حضور دارد، عدد ۱ و در غیر این صورت ۰ می‌نویسیم.

برای پاسخ به پرسمان Brutus AND Caesar AND NOT Calpurnia، کافی است بردارهای بولی مربوط به این سه واژه را برداشته و یک عملیات بیتی روی آن‌ها انجام دهیم:

$$[1, 1, 0, 1, 0, 0] \wedge [1, 1, 0, 1, 1, 1] \wedge [1, 0, 1, 1, 1, 1] = [1, 0, 0, 1, 0, 0]$$

نتیجه این محاسبه به ما می‌گوید که سندهای شماره ۱ (هملت) و ۳ (مکبث) پاسخ‌های درست هستند. این روش، در تئوری بی‌نقص است، اما در عمل با یک مشکل اساسی روبرو است: **اندازه!**

❌ فاجعه مقیاس: ماتریس بیش از حد بزرگ!

فرض کنید مجموعه ما فقط یک میلیون سند با ۵۰۰ هزار واژه منحصر به فرد دارد. ماتریس ما ابعاد $500,000 \times 1,000,000$ خواهد داشت! این ماتریس حدود ۵۰۰ میلیارد درایه دارد. حتی اگر هر درایه فقط یک بایت باشد، به بیش از ۴۶۰ گیگابایت فضا نیاز دارد. مشکل بزرگ‌تر این است که این ماتریس فوق‌العاده **تنگ (sparse)** است. چون هر سند فقط حدود ۱۰۰۰ واژه دارد، از ۵۰۰ میلیارد درایه، کمتر از ۱ میلیارد آن‌ها ۱ هستند و بقیه (یعنی ۹۹.۸٪) همه صفر هستند! ما در حال ذخیره یک اقیانوس از صفرها هستیم!

💡 راه حل نهایی: ایندکس معکوس (نگاهی از زاویه‌ای دیگر)

به جای پرسیدن «کدام اسناد شامل واژه X هستند؟»، مسئله را برعکس می‌کنیم و می‌پرسیم: «واژه X در کدام اسناد ظاهر شده است؟» این تغییر دیدگاه ساده، به **ایندکس معکوس (Inverted Index)** منجر می‌شود.

این ساختار هوشمندانه از دو بخش اصلی تشکیل شده است:

1. **دیکشنری (Dictionary):** یک فهرست مرتب‌شده از تمام واژه‌هایی که در مجموعه وجود دارند. این مانند فهرست مطالب کتاب است.

2. **لیست ارسال (Postings List):** برای هر واژه در دیکشنری، لیستی از تمام اسنادی که آن واژه در آن‌ها یافت شده است. این لیست‌ها معمولاً بر اساس شماره سند (docID) مرتب هستند.

برای مثال ما، ایندکس معکوس به این شکل خواهد بود:

Brutus → [1, 2, 4] | Caesar → [1, 2, 4, 5, 6] | Calpurnia → [2]

حالا برای پاسخ به پرسمان، دیگر نیازی به محاسبات پیچیده و ماتریس عظیم نداریم. فقط کافی است لیست ارسال «بروتوس» و «سزار» را گرفته و اشتراکشان را محاسبه کنیم، و سپس از نتیجه، سند شماره ۲ (که در لیست ارسال «کالپورنیا» است) را حذف کنیم. این فرآیند فوق‌العاده سریع و بهینه است.

🖼️ تصویر نهایی: لحظه «آها!»

تصویر زیر (شکل ۱.۲ در کتاب) دقیقاً همان نتیجه‌ای است که از روش ایندکس معکوس به دست می‌آید. این تصویر، لحظه‌ای است که تلاش فکری ما به نتیجه ملموس تبدیل می‌شود. این تصویر نشان می‌دهد که اسناد ۱ و ۳ (هملت و مکبث) پاسخ صحیح پرسمان ما هستند، دقیقاً همان چیزی که با محاسبات بیتی به دست آوردیم. این تصویر، پلی است بین تئوری ماتریس و کاربرد عملی ایندکس معکوس.

Antony and Cleopatra, Act III, Scene ii
Agrippa [Aside to Domitius Enobarbus]: Why, Enobarbus,
When Antony found Julius Caesar dead,
He cried almost to roaring; and he wept
When at Philippi he found Brutus slain.

Hamlet, Act III, Scene ii
Lord Polonius: I did enact Julius Caesar: I was killed i' the
Capitol; Brutus killed me.

► **Figure 1.2** Results from Shakespeare for the query Brutus AND Caesar AND NOT Calpurnia.

ایدهٔ ایندکس معکوس، سنگ بنای بسیاری از سیستم‌های بازیابی اطلاعات مدرن، از گوگل گرفته تا جستجوی فایل‌های محلی شما، است. شما در حال یادگیری یکی از بنیادی‌ترین و قدرتمندترین مفاهیم در دنیای دیجیتال هستید. 🚀

```
In [1]: ##### اگر یکسری از جاهای کد را متوجه نمیشید، نگران نباشید چون در ادامه‌ی جزوه در مورد تمام کارایی که اینجا داریم میکنیم بیشتر صحبت میکنیم #####

# --- تعریف ساختار داده‌ها ---

# نمونه‌ای از مجموعه اسناد (در اینجا: نمایشنامه‌های شکسپیر)
documents = {
    "Antony": "Agrippa [Aside to Domitius Enobarbus]: Why, Enobarbus, When Antony found Julius Caesar dead, He cried almost to roaring; and he wept When a",
    "Hamlet": "Lord Polonius: I did enact Julius Caesar: I was killed i' the Capitol; Brutus killed me.",
    "Othello": "Had I but one hour tonight-- Othello, I am not well.",
    "Macbeth": "If it were done when 'twere done, then 'twere well it were done quickly."
}
print(documents)
print()

# --- پیاده‌سازی الگوریتم‌ها ---

def build_term_document_matrix(docs):
    """
    را می‌سازد (term-document incidence matrix) این تابع ماتریس وقوع بولی
    خروجی: یک دیکشنری که کلیدها واژه‌ها و مقادیر دیکشنری‌های دیگر هستند
    """
    print("مرحله ۱: ساخت ماتریس وقوع بولی")
    all_terms = set()
    # استخراج تمام واژه‌های یکتا از همه اسناد
    for doc_name, content in docs.items():
        all_terms.update(content.lower().split())

    # ساخت ماتریس با مقادیر اولیه صفر
    term_doc_matrix = {term: {} for term in all_terms}

    # پر کردن ماتریس بر اساس وجود واژه در هر سند
    for term in all_terms:
        for doc_name, content in docs.items():
            term_doc_matrix[term][doc_name] = 1 if term in content.lower() else 0

    print("ماتریس وقوع بولی ساخته شد.")
    return term_doc_matrix

def boolean_query(matrix, query):
    """
    این تابع یک پرسمان بولی را بر اساس ماتریس وقوع اجرا می‌کند.
    مثال: ['Brutus', 'AND', 'Caesar', 'AND', 'NOT', 'Calpurnia']) ورودی: ماتریس وقوع و پرسمان
    """
    print(f"مرحله ۲: اجرای پرسمان بولی")

    # نرمال‌سازی پرسمان: تبدیل واژه‌ها به حروف کوچک (اما نه عملگرها)
    normalized_query = []
    for token in query:
        if token in ['AND', 'OR', 'NOT']:
            normalized_query.append(token)
        else:
            normalized_query.append(token.lower()) # واژه‌ها را کوچک می‌کنیم

    # جداسازی پرسمان به بخش‌های مثبت و منفی
    positive_terms = []
    negative_terms = []
    i = 0
    while i < len(normalized_query):
        token = normalized_query[i]
        if token == 'NOT':
            if i + 1 < len(normalized_query) and normalized_query[i + 1] not in ['AND', 'OR', 'NOT']:
                negative_terms.append(normalized_query[i + 1])
            i += 2
        else:
            positive_terms.append(token)
            i += 1
```

```
        continue
    elif token not in ['AND', 'OR']:
        positive_terms.append(token)
    i += 1

# اگر واژه‌ای وجود نداشت، هیچ سندی برگردانده نشود
if not positive_terms:
    print("نتیجه پرسیمان: []")
    return []

# شروع با اسناد حاوی اولین واژه مثبت
first_term = positive_terms[0]
if first_term not in matrix:
    result_docs = set()
else:
    result_docs = set(doc for doc, exists in matrix[first_term].items() if exists)

# برای سایر واژه‌های مثبت AND اعمال
for term in positive_terms[1:]:
    if term in matrix:
        term_docs = set(doc for doc, exists in matrix[term].items() if exists)
        result_docs = result_docs.intersection(term_docs)
    else:
        result_docs = set() # نتیجه خالی → واژه‌ای نیست
        break

# اعمال NOT
for term in negative_terms:
    if term in matrix:
        term_docs = set(doc for doc, exists in matrix[term].items() if exists)
        result_docs = result_docs.difference(term_docs)

print(f"نتیجه پرسیمان: {sorted(result_docs)}")
return sorted(result_docs)
```

```
def build_inverted_index(matrix):
    """
    این تابع یک ایندکس معکوس از روی ماتریس وقوع می‌سازد.
    این تابع، پل ارتباطی بین ماتریس و ایندکس معکوس را نشان می‌دهد.
    """

    print("\nمرحله ۳: ساخت ایندکس معکوس از ماتریس\n")
    inverted_index = {}

    for term, doc_dict in matrix.items():
        # استخراج اسنادی که واژه در آنها وجود دارد
        docs_with_term = [doc for doc, exists in doc_dict.items() if exists]
        if docs_with_term:
            inverted_index[term] = sorted(docs_with_term)

    print("ایندکس معکوس ساخته شد.")
    # print(inverted_index)
    return inverted_index
```

--- اجرای مثال ---

```
if __name__ == "__main__":
    # مرحله ۱: ساخت ماتریس وقوع بولی
    term_doc_matrix = build_term_document_matrix(documents)

    # نمایش ماتریس برای چند واژه نمونه
    sample_terms = ['Brutus', 'Caesar', 'Calpurnia']
    print(f"نمونه‌ای از ماتریس وقوع برای واژه‌های\n{sample_terms}:")
    for term in sample_terms:
        if term in term_doc_matrix:
            print(f"{term}: {term_doc_matrix[term]}")

    # مرحله ۲: اجرای پرسیمان بولی
    query = ['Brutus', 'AND', 'Caesar', 'AND', 'NOT', 'Calpurnia']
    result = boolean_query(term_doc_matrix, query)

    # مرحله ۳: ساخت ایندکس معکوس
    inverted_index = build_inverted_index(term_doc_matrix)

    # نمایش ساختار ایندکس معکوس
    sample_terms = ['Brutus', 'Caesar', 'Calpurnia']
    print(f"نمونه‌ای از ماتریس وقوع برای واژه‌های\n{sample_terms}:")
    for term in sample_terms:
        lower_term = term.lower()
        if lower_term in inverted_index:
            print(f"{term}: {inverted_index[lower_term]}")
        else:
            print(f"{term}: (وجود ندارد در ماتریس)")

    print("\nاین فرایند، چگونگی تبدیل یک مسئله جستجوی خطی به یک رامحل بهینه (ایندکس معکوس) را نشان می‌دهد.")
```

{ 'Antony': 'Agrippa [Aside to Domitius Enobarbus]: Why, Enobarbus, When Antony found Julius Caesar dead, He cried almost to roaring; and he wept When at Philippi he found Brutus slain.', 'Hamlet': "Lord Polonius: I did enact Julius Caesar: I was killed i' the Capitol; Brutus killed me.", 'Othello': 'Had I but one hour tonight-- Othello, I am not well.', 'Macbeth': "If it were done when 'twere done, then 'twere well it were done quickly."}

...مرحله ۱: ساخت ماتریس وقوع بولی
ماتریس وقوع بولی ساخته شد

['Brutus', 'Caesar', 'Calpurnia']: نمونه‌ای از ماتریس وقوع برای واژه‌های

Brutus AND Caesar AND NOT Calpurnia :مرحله ۲: اجرای پرسمان بولی
نتیجه پرسمان ['Antony', 'Hamlet']

...مرحله ۳: ساخت ایندکس معکوس از ماتریس
ایندکس معکوس ساخته شد

['Brutus', 'Caesar', 'Calpurnia']: نمونه‌ای از ماتریس وقوع برای واژه‌های
Brutus: {'Antony': 1, 'Hamlet': 1, 'Othello': 0, 'Macbeth': 0}
Caesar: {'Antony': 1, 'Hamlet': 1, 'Othello': 0, 'Macbeth': 0}
Calpurnia: (وجود ندارد در ماتریس)

این فرآیند، چگونگی تبدیل یک مسئله جستجوی خطی به یک رامحل بهینه (ایندکس معکوس) را نشان می‌دهد

🔍 1.2 ساخت اولیه فهرست معکوس: پشت صحنه جادوی جستجو!

🚀 یک چالش دنیای واقعی

تصور کنید مدیر داده یک شبکه اجتماعی بزرگ مانند «Instagram» هستید. میلیون‌ها پست روزانه منتشر می‌شود. یک کاربر می‌خواهد تمام پست‌هایی را ببیند که همزمان دارای هشتگ‌های #سفر و #غروب_آفتاب هستند. چگونه می‌توانید این کار را در کسری از ثانیه انجام دهید؟ پیمایش تمام میلیون‌ها پست برای هر جستجو غیرممکن است! اینجااست که جادوی **فهرست معکوس** وارد می‌شود.

برای بهره‌گیری از سرعت فوق‌العاده در زمان جستجو، باید این فهرست را از قبل بسازیم. این فرآیند مانند تهیه یک دستور آشپزی دقیق است و چهار مرحله اصلی دارد:

1. جمع‌آوری مواد اولیه (اسناد)

همه پست‌های IG را جمع می‌کنیم. این «مجموعه اسناد» ماست.

2. خرد کردن (توکن‌سازی)

هر پست را به واحدهای کوچک‌تری به نام «توکن» (Token) می‌شکنیم. در دنیای ما، این یعنی جدا کردن تمام کلمات و هشتگ‌ها. 🌿

3. تمیزکاری و آماده‌سازی (پیش‌پردازش زبانی)

توکن‌ها را نرمال‌سازی می‌کنیم: همه را به حروف کوچک تبدیل می‌کنیم، علائم نگارشی مانند «!» و «#» را حذف می‌کنیم و ریشه کلمات را پیدا می‌کنیم. 🧠

4. ساخت دفترچه تلفن هوشمند (ایندکس‌سازی)

حالا مهم‌ترین بخش کار: ساختن خود فهرست معکوس. 📖

📖 جادوی مرتب‌سازی: از آشفتگی تا نظم

چگونه این دفترچه تلفن را می‌سازیم؟ روش اولیه و هوشمندانه، **ایندکس‌سازی مبتنی بر مرتب‌سازی** است:

مرحله الف: آشفت‌گاه اولیه

ابتدا یک لیست بزرگ از تمام جفت‌های (توکن، شناسه_پست) برای کل مجموعه اسناد می‌سازیم. مثلاً:

(sunset, 1542)

(travel, 1123)

(سفر, 1123)

(good, 8801)

(غروب_آفتاب, 1542)

...

مرحله ب: نظم جادویی

حالا این لیست شلوغ را بر اساس توکن‌ها (الفبا) مرتب می‌کنیم. سپس موارد تکراری را گروه‌بندی می‌کنیم. نتیجه چیزی شبیه به این می‌شود:

```
good → 8801
sunset → 1542
travel → 1123
سفر → 1123
غروب_آفتاب → 1542
...
```

این ساختار مرتب، قلب تپنده فهرست معکوس ماست.

در نهایت، این دفترچه تلفن هوشمند به دو بخش اصلی تقسیم می‌شود:

📁 دو بخش اصلی فهرست معکوس

- دیکشنری (Dictionary):** این بخش مانند فهرست نام‌ها در دفترچه تلفن است. شامل تمام توکن‌های منحصر به فرد (مانند سفر، غروب_آفتاب) است. برای هر توکن، یک آمار کلیدی هم ذخیره می‌شود: **فراوانی سند** (Document Frequency یا df_t)؛ یعنی تعداد کل پست‌هایی که آن توکن در آن‌ها وجود دارد. این دیکشنری آنقدر کوچک است که می‌توانیم آن را کاملاً در حافظه اصلی (RAM) نگه داریم تا دسترسی به آن فوری باشد.
- لیست ارسال (Postings List):** این بخش مانند شماره تلفن‌های مربوط به هر نام در دفترچه تلفن است. برای هر توکن در دیکشنری، لیستی از شناسه‌های پست‌هایی (docID) که آن توکن در آن‌ها وجود دارد، ذخیره می‌شود. این لیست‌ها بسیار حجیم هستند و روی دیسک ذخیره می‌شوند، اما **مرتب‌شده بر اساس docID** هستند. این مرتب‌سازی، کلید سرعت پردازش پرسman‌های بولی است که در بخش قبل دیدیم.

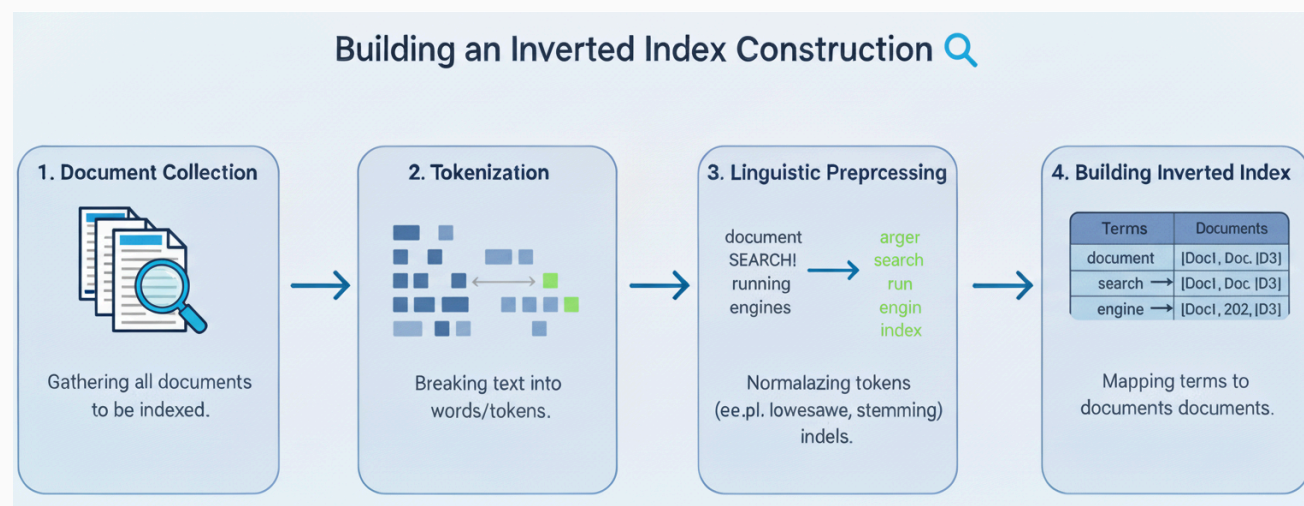
🤖 عمیق‌تر: بهترین ساختار برای لیست ارسال چیست؟

برای ذخیره لیست ارسال در حافظه، دو گزینه اصلی وجود دارد و انتخاب بین آن‌ها یک معامله بین انعطاف‌پذیری، سرعت و فضا است:

- لیست پیوندی تک‌طرفه (Singly Linked List):** مثل قطاری از واگن‌ها که به هم متصل‌اند. **مزیت:** اضافه کردن یک پست جدید به لیست بسیار سریع و ارزان است. **عیب:** به حافظه پراکنده نیاز دارد و پیمایش آن ممکن است کندتر باشد. این ساختار، پایه‌ای برای تکنیک‌های پیشرفته‌تر مانند *Skip Lists* است.
- آرایه با طول متغیر (Variable Length Array):** مثل یک قطار یکپارچه با تعداد واگن متغیر. **مزیت:** بسیار فشرده‌تر است و چون در حافظه پشت سر هم قرار گرفته، پیمایش آن در پردازنده‌های مدرن بسیار سریع‌تر است. **عیب:** اضافه کردن عضو جدید ممکن است نیازمند ساختن کل آرایه جدیدی باشد.

در دنیای واقعی، اگر به‌روزرسانی زیاد باشد، از لیست پیوندی استفاده می‌کنیم. اما اگر اسناد کمتر تغییر کنند، آرایه‌ها به دلیل فشردگی و سرعت بالا، برنده هستند! 📁

این ساختار به ظاهر ساده، **بی‌رقیب‌ترین و کارآمدترین روش** برای پشتیبانی از جستجوی متنی موردی (ad-hoc) است. هر بار که در گوگل عبارتی را جستجو می‌کنید یا در یک پایگاه داده حقوقی به دنبال یک سند خاص می‌گردید، در حال استفاده از نسخه‌ای بسیار پیچیده‌تر از همین ایده ساده و زیبا هستید.



1.3 پردازش پرسمان‌های بولی

🎓 داستان‌ی از کلاس درس

"روز اول کلاس بازیابی اطلاعات بود. دکتر اسکندری وارد شد و پرسید: 'چطور می‌توانم در این کتابخانه‌های دیجیتال، کتاب‌هایی را پیدا کنم که هم درباره‌ی هوش مصنوعی و هم درباره‌ی VLM‌ها باشند؟' سکوت حاکم شد. سپس دکتر لبخندی زد و گفت: 'این همان چیزی است که امروز می‌خواهیم یاد بگیریم - جادوی پرسمان‌های بولی!'"

در مدل بولی، هر پرسمان ترکیبی از واژه‌ها با عملگرهای AND، OR و NOT است. این مدل ساده اما قدرتمند، پایه‌ی بسیاری از موتورهای جستجوی اولیه بود و هنوز هم در سیستم‌های اطلاعاتی مدرن کاربرد دارد.

💡 فکر کن!

تا حالا به این فکر کرده‌اید که چطور گوگل نتایج جستجو را فیلتر می‌کند؟ وقتی تایپ می‌کنید "رستوران ایتالیایی نزدیک من" AND "باز تا ساعت ۱۰ شب"، در حال استفاده از منطق بولی هستید! 🍕 🌙

برای پاسخ به پرسمان ساده‌ای مانند:

Brutus $\wedge$ Calpurnia

مراحل زیر را دنبال می‌کنیم:

- 🔍 یافتن Brutus در دیکشنری
- 📋 بازیابی لیست ارسال آن
- 🔍 یافتن Calpurnia در دیکشنری
- 📋 بازیابی لیست ارسال آن
- 🔗 محاسبهٔ اشتراک دو لیست — یعنی یافتن اسنادی که هر دو واژه در آن‌ها ظاهر شده‌اند

🎬 تصور کن...

دو لیست شماره تلفن داریم: یکی از دوستان به نام "علی" و دیگری از دوستان که در "تهران" زندگی می‌کنند. می‌خواهی بدانی کدام علی‌ها در تهران زندگی می‌کنند. چه کار می‌کنی؟ دقیقاً! هر دو لیست را نگاه می‌کنی و شماره‌هایی که در هر دو

چون لیست‌های ارسال مرتب‌شده بر اساس docID هستند، می‌توانیم از الگوریتم ادغام دوطرفه (two-pointer merge) برای یافتن اشتراک در زمان خطی استفاده کنیم. این الگوریتم در شکل ۱.۶ کتاب آمده است:

```
INTERSECT( $p_1, p_2$ )
1   $answer \leftarrow \langle \rangle$ 
2  while  $p_1 \neq \text{NIL}$  and  $p_2 \neq \text{NIL}$ 
3  do if  $\text{docID}(p_1) = \text{docID}(p_2)$ 
4      then  $\text{ADD}(answer, \text{docID}(p_1))$ 
5           $p_1 \leftarrow \text{next}(p_1)$ 
6           $p_2 \leftarrow \text{next}(p_2)$ 
7  else if  $\text{docID}(p_1) < \text{docID}(p_2)$ 
8      then  $p_1 \leftarrow \text{next}(p_1)$ 
9  else  $p_2 \leftarrow \text{next}(p_2)$ 
10 return  $answer$ 
```

► Figure 1.6 Algorithm for the intersection of two postings lists p_1 and p_2 .

🚀 تحلیل پیچیدگی

پیچیدگی زمانی این الگوریتم برابر $O(x + y)$ است، که x و y طول لیست‌های ارسال هستند. این بسیار سریع‌تر از اسکن خطی تمام مجموعه است، هرچند از نظر نماد Θ هم‌مرتبه باشد – اما ثابت عملی آن بسیار کوچک‌تر است.

تفکر سریع: اگر لیست اول ۱۰۰۰ آیتم و لیست دوم ۵۰۰ آیتم داشته باشد، در بدترین حالت چند مقایسه نیاز داریم؟ 🤔

◀ پاسخ

برای پرسمان‌های چندجمله‌ای مانند:

$\text{Brutus} \wedge \text{Caesar} \wedge \text{Calpurnia}$

بهینه‌ترین استراتژی، پردازش واژه‌ها به ترتیب فراوانی صعودی است: ابتدا واژه‌ای که در کمترین تعداد سند آمده را بررسی می‌کنیم، چون نتیجه میانی از همان ابتدا کوچک خواهد بود و کار کمتری برای اشتراک بعدی لازم است. برای مثال، اگر:

• Calpurnia: 2 سند

• Brutus: 174 سند

• Caesar: 132 سند

🎯 استراتژی بهینه

آنگاه بهتر است پرسمان را به صورت زیر ارزیابی کنیم:

$(\text{Calpurnia} \wedge \text{Brutus}) \wedge \text{Caesar}$

🏠 مثال کاربردی

تصور کنید در یک فروشگاه آنلاین کار می‌کنید و مشتری به دنبال محصولی است که همزمان این ویژگی‌ها را داشته باشد: "قیمت کمتر از ۱۰۰ هزار تومان" AND "رنگ آبی" AND "برند X". اگر بدانید که فقط ۵ محصول رنگ آبی دارند، ۵۰ محصول

برند X هستند و ۲۰۰ محصول قیمت کمتر از ۱۰۰ هزار تومان دارند، از کدام فیلتر شروع می‌کنید؟ طبیعتاً از رنگ آبی چون نتایج اولیه را به ۵ محصول محدود می‌کند! 🛒

این تکنیک، **بهینه‌سازی پرسمان** (Query Optimization) نام دارد و نیازمند ذخیرهٔ **فراوانی سند** (df_t) در دیکشنری است.

برای پرسمان‌های پیچیده‌تر مانند:

$$\begin{aligned} &(\text{Brutus} \vee \text{Caesar}) \\ &\wedge \neg(\text{Antony} \vee \text{Cleopatra}) \end{aligned}$$

ابتدا هر زیرعبارت را ارزیابی می‌کنیم:

• Brutus OR Caesar: ادغام دو لیست (UNION)

• Antony OR Cleopatra: ادغام دو لیست دیگر

• NOT (...): مکمل گرفتن نسبت به تمام اسناد مجموعه

⚠ چالش NOT

اما **پردازش NOT مشکل‌ساز است**، چون نیاز به دانستن مجموعهٔ کامل اسناد دارد – و اگر مجموعه بزرگ باشد، ساخت مکمل بسیار هزینه‌بر است. راه‌حل‌های هوشمندانه‌تری (مثل استفاده از ایندکس معکوس معکوس!) نیاز است، که در اینجا به‌صورت ساده پیاده‌سازی می‌شود.

تقویت درک: تصور کنید در یک مهمانی هستید و می‌خواهید کسانی را پیدا کنید که "نوشیدنی نمی‌خورند". آیا راحت‌تر است همه‌ی مهمان‌ها را بررسی کنید یا لیست کسانی را که "نوشیدنی می‌خورند" تهیه کرده و از کل لیست مهمان‌ها کم کنید؟ 🍹

الگوریتم کلی برای پرسمان‌های ترکیبی در شکل ۱.۷ کتاب آمده است – که واژه‌ها را بر اساس فراوانی مرتب کرده و به‌صورت تدریجی اشتراک می‌گیرد. این، پایهٔ اصلی موتورهای جست‌وجوی بولی است.

```
INTERSECT( $\langle t_1, \dots, t_n \rangle$ )
1   $terms \leftarrow \text{SORTBYINCREASINGFREQUENCY}(\langle t_1, \dots, t_n \rangle)$ 
2   $result \leftarrow \text{postings}(\text{first}(terms))$ 
3   $terms \leftarrow \text{rest}(terms)$ 
4  while  $terms \neq \text{NIL}$  and  $result \neq \text{NIL}$ 
5  do  $result \leftarrow \text{INTERSECT}(result, \text{postings}(\text{first}(terms)))$ 
6      $terms \leftarrow \text{rest}(terms)$ 
7  return  $result$ 
```

► **Figure 1.7** Algorithm for conjunctive queries that returns the set of documents containing each term in the input list of terms.

🌐 کاربردهای امروزی

اگرچه مدل بولی ساده به نظر می‌رسد، اما هنوز در بسیاری از سیستم‌های مدرن استفاده می‌شود. برای مثال:

- در پایگاه‌های داده حقوقی برای جستجوی دقیق قوانین و احکام
- در جستجوی اسناد علمی برای یافتن مقالاتی که همزمان چند مفهوم را پوشش می‌دهند
- در ویژگی‌های جستجوی پیشرفته گوگل (مانند جستجوی عبارت دقیق با استفاده از کوتیشن)
- در سیستم‌های توصیه‌گر برای فیلتر کردن محتوا بر اساس علایق کاربران

حالا که با مفاهیم آشنا شدی، بیا یک سناریوی واقعی را حل کنیم:

شما توسعه‌دهنده یک اپلیکیشن دوست‌یابی هستید. کاربر می‌خواهد فردی را پیدا کند که این ویژگی‌ها را داشته باشد: "علاقه‌مند به سفر" AND "علاقه‌مند به کتاب" AND NOT "سیگار می‌کشد". با توجه به آمار زیر، بهینه‌ترین استراتژی برای جستجو چیست؟

- علاقه‌مند به سفر: ۱۰,۰۰۰ کاربر - علاقه‌مند به کتاب: ۵,۰۰۰ کاربر - سیگار می‌کشد: ۲,۰۰۰ کاربر

راهنمایی

پاسخ

📖 برای مطالعه بیشتر

اگر به این موضوع علاقه‌مند شدی، پیشنهاد می‌کنم مقاله "Beyond Boolean: The Evolution of Information Retrieval Models" را مطالعه کنی. این مقاله نشان می‌دهد که چگونه مدل‌های بولی به مدل‌های پیچیده‌تر مانند مدل فضای برداری و مدل‌های احتمالی تبدیل شده‌اند. 📖

In [18]: # --- تعریف ساختار داده‌ها ---

```
# نمونه‌ای از یک ایندکس معکوس (ساده‌شده)
# مرتب‌شده هستند (postings lists) کلیدها وازه‌ها و مقادیر، لیست‌های ارسال
# این ساختار، قلب یک موتور جستجوی بولی است.
inverted_index = {
    "Brutus": [1, 2, 4, 11, 31, 45, 173, 174],
    "Calpurnia": [2, 31, 54, 101],
    "Caesar": [1, 2, 4, 5, 6, 16, 57, 132],
    "Antony": [5, 10, 31],
    "Cleopatra": [2, 3, 31, 64]
}

# فرض می‌کنیم مجموعه اسناد ما شامل شناسه‌های ۱ تا ۲۰۰ است.
# ضروری است تا بتوانیم مکمل یک مجموعه را محاسبه کنیم NOT این لیست برای عملگر
ALL_DOCUMENTS = list(range(1, 201))
```

--- پیاده‌سازی الگوریتم‌های اصلی ---

```
def intersect(p1, p2):
    """
    الگوریتم ادغام دوطرفه برای یافتن اشتراک دو لیست مرتب‌شده.
    این پیاده‌سازی دقیقاً مطابق با شبه‌کد کتاب و توضیحات مارک‌دان سل است.
    عمل می‌کند O(len(p1) + len(p2)) این الگوریتم به صورت خطی و با حداکثر کارایی
    """
    print(f"{p1} و {p2}: در حال محاسبه اشتراک لیست‌ها")
    i, j = 0, 0 # دو نشانگر (pointer)
    result = []
    # تا زمانی که هر دو نشانگر به انتها نرسیده‌اند، به مقایسه ادامه می‌دهیم
    while i < len(p1) and j < len(p2):
        # اگر آیتم‌ها برابر بودند، در نتیجه ذکر شده و هر دو نشانگر جلو می‌روند
        if p1[i] == p2[j]:
            result.append(p1[i])
            i += 1
            j += 1
        # اگر آیتم لیست اول کوچکتر بود، فقط نشانگر اول را جلو می‌بریم
        elif p1[i] < p2[j]:
            i += 1
        # در غیر این صورت، نشانگر دوم را جلو می‌بریم
        else:
            j += 1
    print(f"نتیجه اشتراک: {result}")
    return result
```

```
def union(p1, p2):
    """
    الگوریتم ادغام برای یافتن اجتماع دو لیست مرتب‌شده (بدون تکرار).
    استفاده می‌شود OR این تابع برای پردازش عملگر
    """
    print(f"{p1} و {p2}: در حال محاسبه اجتماع لیست‌ها")
    i, j = 0, 0
    result = []
    # تا زمانی که هر دو لیست آیتم دارند
```

```
while i < len(p1) and j < len(p2):
    if p1[i] == p2[j]:
        result.append(p1[i])
        i += 1
        j += 1
    elif p1[i] < p2[j]:
        result.append(p1[i])
        i += 1
    else:
        result.append(p2[j])
        j += 1
# اضافه کردن آیتم‌های باقی‌مانده در هر دو لیست
result.extend(p1[i:])
result.extend(p2[j:])
print(f"نتیجه اجتماع: {result}")
return result
```

```
def complement(postings_list):
    """
    برای پیاده‌سازی عملگر نفی (NOT) تابع مکمل
    این عملیات هزینه‌بر است چون به کل مجموعه اسناد نیاز دارد
    """

    print(f"در حال محاسبه مکمل لیست: { postings_list}")
    p1_iter = iter(postings_list)
    current_doc_to_exclude = next(p1_iter, None)
    result = []
    for doc_id in ALL_DOCUMENTS:
        if current_doc_to_exclude is not None and doc_id == current_doc_to_exclude:
            current_doc_to_exclude = next(p1_iter, None)
        else:
            result.append(doc_id)
    # برای نمایش بهتر، فقط ۱۰ سند اول نتیجه را چاپ می‌کنیم
    print(f"{result[:10]}...")
    return result
```

```
def get_postings(term):
    """تابع کمکی برای گرفتن لیست ارسال یک واژه"""
    return inverted_index.get(term, [])
```

```
def process_conjunctive_query(terms):
    """
    با بهینه‌سازی (conjunctive query) پردازش یک پرسمان ترکیبی
    واژه‌ها بر اساس فراوانی (طول لیست ارسال) به ترتیب صعودی پردازش می‌شوند
    این مهم‌ترین تکنیک بهینه‌سازی برای پرسمان‌های بولی است
    """

    print(f"\n--- پردازش پرسمان AND: {' AND ' .join(terms)} ---")

    # مرحله ۱: مرتب‌سازی واژه‌ها بر اساس طول لیست ارسال (بهینه‌سازی کلیدی)
    terms_with_freq = [(term, len(get_postings(term))) for term in terms]
    terms_with_freq.sort(key=lambda x: x[1])
    sorted_terms = [term for term, freq in terms_with_freq]
    print(f"{sorted_terms}: ترتیب بهینه پردازش واژه‌ها (بر اساس فراوانی صعودی)")

    # مرحله ۲: شروع با کوچکترین لیست و اشتراک گرفتن تدریجی
    result = get_postings(sorted_terms[0])
    print(f"{sorted_terms[0]}' : {result}")

    for term in sorted_terms[1:]:
        if not result: # دیگر ادامه نده
            print("نتیجه میانی خالی شد، پردازش متوقف شد")
            break
        result = intersect(result, get_postings(term))

    print(f"نتیجه نهایی پرسمان: {result}")
    return result
```

--- اجرای مثال‌ها ---

```
if __name__ == "__main__":

    # مثال ۱: پرسمان ساده (Brutus AND Calpurnia)
    print("="*50)
    print("مثال ۱: پرسمان ساده")
    brutus_postings = get_postings("Brutus")
    calpurnia_postings = get_postings("Calpurnia")
    intersect(brutus_postings, calpurnia_postings)

    # مثال ۲: پرسمان ترکیبی با بهینه‌سازی (Brutus AND Caesar AND Calpurnia)
    print("\n" + "="*50)
    print("مثال ۲: پرسمان ترکیبی با بهینه‌سازی")
    process_conjunctive_query(["Brutus", "Caesar", "Calpurnia"])

    # NOT و OR مثال ۳: پرسمان پیچیده با
    print("\n" + "="*50)
    print("NOT و OR مثال ۳: پرسمان پیچیده با")
    # (Brutus OR Caesar) AND NOT (Antony OR Cleopatra)

    # مرحله ۱: محاسبه (Brutus OR Caesar)
    brutus_caesar_union = union(get_postings("Brutus"), get_postings("Caesar"))

    # مرحله ۲: محاسبه (Antony OR Cleopatra)
    antony_cleopatra_union = union(get_postings("Antony"), get_postings("Cleopatra"))

    # NOT مرحله ۳: محاسبه (Antony OR Cleopatra)
    # هشدار: این عملیات بسیار سنگین است
    # antony_cleopatra_complement = complement(antony_cleopatra_union)

    # مرحله ۴: محاسبه نهایی (اشتراک)
    # intersect(brutus_caesar_union, antony_cleopatra_complement) = نتیجه نهایی
    # آن را کامنت می‌کنیم، NOT برای جلوگیری از اجرای عملیات سنگین
    print("محاسبه نهایی انجام نمی‌شود، NOT برای جلوگیری از عملیات سنگین")
    print("استفاده می‌کنند NOT در عمل، موتورهای جستجو از تکنیک‌های هوشمندانه‌تری برای پردازش")
```

=====

مثال ۱: پرسمان ساده

در حال محاسبه اشتراک لیست‌ها: [1, 2, 4, 11, 31, 45, 173, 174] و [2, 31, 54, 101]

نتیجه اشتراک: [2, 31]

=====

مثال ۲: پرسمان ترکیبی با بهینه‌سازی

--- AND: Brutus AND Caesar AND Calpurnia ---

['Calpurnia', 'Brutus', 'Caesar'] :ترتیب بهینه پردازش واژه‌ها (بر اساس فراوانی صعودی)

'Calpurnia': [2, 31, 54, 101] شروع با لیست کوچکترین واژه

در حال محاسبه اشتراک لیست‌ها: [2, 31, 54, 101] و [1, 2, 4, 11, 31, 45, 173, 174]

نتیجه اشتراک: [2, 31]

در حال محاسبه اشتراک لیست‌ها: [2, 31] و [1, 2, 4, 5, 6, 16, 57, 132]

نتیجه اشتراک: [2]

نتیجه نهایی پرسمان: [2]

=====

NOT و OR مثال ۳: پرسمان پیچیده با

در حال محاسبه اجتماع لیست‌ها: [1, 2, 4, 11, 31, 45, 173, 174] و [1, 2, 4, 5, 6, 16, 57, 132]

نتیجه اجتماع: [1, 2, 4, 5, 6, 11, 16, 31, 45, 57, 132, 173, 174]

در حال محاسبه اجتماع لیست‌ها: [5, 10, 31] و [2, 3, 31, 64]

نتیجه اجتماع: [2, 3, 5, 10, 31, 64]

.محاسبه نهایی انجام نمی‌شود، NOT برای جلوگیری از عملیات سنگین

.استفاده می‌کنند NOT در عمل، موتورهای جستجو از تکنیک‌های هوشمندانه‌تری برای پردازش

1.4 مدل بولی گسترش‌یافته در مقابل بازیابی رتبه‌بندی‌شده

دو دستیار در ذهن شما تصور کنید

دستیار اول بسیار دقیق و خشک است. اگر بگویید «قهوه و کیک»، دقیقاً اسنادی را پیدا می‌کند که هر دو کلمه را داشته باشند، نه یکی بیشتر. این دستیار، **مدل بولی** است.

دستیار دوم هوشمند و شهودی است. اگر بگویید «یک کافه دنج برای عصر جمعه»، او با توجه به کل جملات، مفهوم «دنج» و «عصر جمعه» را می‌فهمد و بهترین گزینه‌ها را به شما پیشنهاد می‌دهد، حتی اگر دقیقاً این کلمات در متن نباشند. این دستیار، **مدل بازیابی رتبه‌بندی‌شده (Ranked Retrieval)** است.

برای دهه‌ها، دنیای جستجوی حرفه‌ای، به‌ویژه در حوزه‌های تخصصی مانند حقوق، تحت سلطه دستیار اول (مدل بولی) بود. اما نسخه‌ی ساده‌ی بولی که با `AND` و `OR` آشنا شدیم، برای کاربران حرفه‌ای بسیار محدود بود. آن‌ها به یک نسخه‌ی **ابرسوپر** نیاز داشتند!

مهم‌ترین ارتقا، اضافه شدن **عملگرهای نزدیکی (Proximity Operators)** بود. این عملگرها به دستیار ما کمک می‌کنند تا بفهمد دو کلمه فقط در یک سند وجود ندارند، بلکه «کنار هم» معنای خاصی ایجاد می‌کنند.

مطالعه موردی: دنیای جستجوی حقوقی (Westlaw)

تصور کنید یک وکیل برجسته هستید و باید پرونده‌ای پیچیده درباره «مسئولیت میزبان در قبال مهمانان مست» را تحقیق کنید. در سیستم Westlaw، شما یک پرسمان غول‌پیکر و دقیق می‌نویسید، نه یک جستجوی ساده:

```
host! /p (responsib! liab!) /p (intoxicat! drunk!) /p guest
```

بیایید این جادو را با هم بشکافیم:

- host!** : هر کلمه‌ای که با **host** شروع شود (مثل host, host's).
- p/** : معجون اصلی! یعنی «در همان **پاراگراف**».
- (responsib! liab!)** : کلماتی مثل responsible یا liability.

این پرسمان می‌گوید: «اسنادی را پیدا کن که در یک پاراگراف، کلماتی از خانواده host، در پاراگراف دیگری کلماتی از خانواده مسئولیت، و در پاراگراف سومی کلماتی از خانواده مستی و مهمان وجود داشته باشد.» این سطح از کنترل، برای یک

اما یک بحث بزرگ همیشه وجود داشته: کدام روش بهتر است؟

🏛️ نبرد دقت (Precision) در مقابل بازیابی (Recall)

مدل بولی یک مشکل اساسی دارد:

- استفاده زیاد از **AND** → نتایج بسیار **مربوط** ولی **کم**. (دقت بالا، بازیابی پایین)
- استفاده زیاد از **OR** → نتایج بسیار **زیاد** ولی بسیاری از آن‌ها **غیرمربوط**. (دقت پایین، بازیابی بالا)

یافتن نقطه‌ی طلایی بین این دو، مثل راه رفتن روی تیغ یک کارد است! به همین دلیل، مطالعات (مانند تحقیق Turtle در ۱۹۹۴) نشان دادند که حتی برای وکلا، پرسمان‌های متنی آزاد (دستیار دوم) اغلب نتایج بهتری تولید می‌کنند.

پس راه حل چیست؟ چگونه می‌توانیم بهترین هر دو دنیا را داشته باشیم؟ اینجااست که ما به **چهار ستون موتورهای جستجوی مدرن** می‌رسیم:

1. 🔍 **جستجوی بخشنده (Tolerant Search):** سیستمی که اشتباهات املایی شما را می‌بخشد و می‌فهمد که «ماشین» و «دستگاه» می‌توانند مترادف باشند.
2. 🌿 **جستجوی بافتار (Proximity & Phrase Search):** سیستمی که تفاوت بین «سگ گاز می‌گیرد» و «انسان به سگ گاز می‌گیرد» را می‌فهمد. این یعنی درک نزدیکی و ترتیب کلمات.
3. 📊 **اهمیت فراوانی (Term Frequency):** سیستمی که می‌داند اگر کلمه «هوش مصنوعی» در یک سند ۱۰ بار تکرار شده، آن سند احتمالاً از سندی که فقط یک بار به آن اشاره کرده، مرتبط‌تر است.
4. 🏆 **هنر رتبه‌بندی (Ranking):** سیستمی که به جای گفتن «بله/خیر»، به هر سند یک امتیاز از ۱۰۰ می‌دهد و بهترین‌ها را در بالای لیست قرار می‌دهد.

🌐 سفر از کتابخانه حقوق تا گوگل

این چهار ستون، فناوری است که امروزه دنیا را تسخیر کرده است. بار بعدی که در گوگل عبارتی را جستجو می‌کنید، به یاد داشته باشید که در پشت این کادر ساده، این ایده‌ها با قدرت در حال کار هستند. گوگل ممکن است ظاهراً یک جستجوی متنی آزاد ارائه دهد، اما در پشت صحنه از جستجوی عبارت (دستور دوم) و نسخه‌های ساده‌شده‌ای از منطق بولی استفاده می‌کند.

In [4]: # --- تعریف ساختار داده‌ها ---

کلاس ساده برای نمایش یک سند با محتوا و پاراگراف‌های آن

```
class Document:
    def __init__(self, doc_id, content):
        self.doc_id = doc_id
        # تقسیم محتوا به پاراگراف‌ها بر اساس خطوط خالی
        self.paragraphs = [p.strip() for p in content.split('\n\n') if p.strip()]
        # ایجاد لیستی از کلمات در هر پاراگراف برای جستجوی سریع‌تر
        self.paragraph_words = [p.lower().split() for p in self.paragraphs]

    def __str__(self):
        return f"سند {self.doc_id}: {self.paragraphs[0][:50]}..."
```

نمونه‌ای از مجموعه اسناد کوچک

```
documents = [
    Document(1, "مهمانان در مهمانی مست بودند. میزبان مسئولیت مواد الکلی را بر عهده داشت. مهمانان پس از ترک مهمانی تصادف کردند"),
    Document(2, "میزبان مهمانی دیگری برگزار کرد. در این مهمانی، مهمانان الکل مصرف نکردند. مسئولیت میزبان در این مورد کمتر بود"),
    Document(3, "قوانین مربوط به مسئولیت میزبان در بسیاری از کشورها وجود دارد. اگر مهمان مست باشد و حادثه‌ای رخ دهد، میزبان مسئول است"),
    Document(4, "مهمان و میزبان توافق‌نامه‌ای قبل از مهمانی امضا کردند. این توافق‌نامه مسئولیت‌ها را مشخص می‌کرد"),
    Document(5, "در این مهمانی، میزبان به مهمانان الکل نداد. با این حال، یکی از مهمانان مشروب خود را آورده بود و مست شد")
]
```

```
# --- پیادسازی الگوریتم‌های جستجو ---

def boolean_search(documents, query_terms):
    """
    جستجوی بولی ساده: اسنادی را برمی‌گرداند که تمام واژه‌های پرسمان را دارند
    """
    results = []
    for doc in documents:
        # تبدیل کل سند به حروف کوچک برای جستجوی حساس‌به‌حروف بزرگ/کوچک نباشیم
        doc_text = ' '.join(doc.paragraphs).lower()
        if all(term.lower() in doc_text for term in query_terms):
            results.append(doc)
    return results

def proximity_search_paragraph(doc, term1, term2):
    """
    بررسی می‌کند که آیا دو واژه در یک پاراگراف از سند وجود دارند یا نه
    است (همان پاراگراف) /p این پیاده‌سازی برای عملگر
    """
    for words in doc.paragraph_words:
        has_term1 = any(term1.lower() in word for word in words)
        has_term2 = any(term2.lower() in word for word in words)
        if has_term1 and has_term2:
            return True
    return False

def extended_boolean_search(documents, term1, term2, operator):
    """
    جستجوی بولی گسترش‌یافته با عملگرهای نزدیکی
    """
    results = []
    for doc in documents:
        if operator == "AND":
            # جستجوی بولی ساده
            doc_text = ' '.join(doc.paragraphs).lower()
            if term1.lower() in doc_text and term2.lower() in doc_text:
                results.append(doc)
        elif operator == "/p":
            # جستجوی نزدیکی در همان پاراگراف
            if proximity_search_paragraph(doc, term1, term2):
                results.append(doc)
    return results

def term_frequency(doc, term):
    """
    تعداد تکرار یک واژه در یک سند را می‌شمارد
    """
    count = 0
    for paragraph in doc.paragraphs:
        words = paragraph.lower().split()
        count += sum(1 for word in words if term.lower() in word)
    return count

def ranked_retrieval(documents, query_terms):
    """
    بازیابی رتبه‌بندی‌شده: اسناد را بر اساس مجموع فراوانی واژه‌های پرسمان رتبه‌بندی می‌کند
    """
    doc_scores = []
    for doc in documents:
        score = sum(term_frequency(doc, term) for term in query_terms)
        if score > 0: # فقط اسنادی که حداقل یکی از واژه‌ها را دارند
            doc_scores.append((doc, score))

    # مرتب‌سازی اسناد بر اساس امتیاز (از بیشترین به کمترین)
    doc_scores.sort(key=lambda x: x[1], reverse=True)
    return doc_scores

# --- اجرای مثال‌ها ---

if __name__ == "__main__":

    # مثال ۱: مقایسه جستجوی بولی ساده و جستجوی نزدیکی
    print("="*60)
    print("مثال ۱: مقایسه جستجوی بولی ساده و جستجوی نزدیکی")
    term1 = "میزبان"
    term2 = "مهمان"

    # جستجوی بولی ساده (AND)
    boolean_results = boolean_search(documents, [term1, term2])
    print(f"نتایج جستجوی بولی ساده({term1} AND {term2}):")
    for doc in boolean_results:
        print(f" - {doc}")

    # جستجوی نزدیکی (/p)
    proximity_results = extended_boolean_search(documents, term1, term2, "/p")
    print(f"نتایج جستجوی نزدیکی({term1} /p {term2}):")
    for doc in proximity_results:
        print(f" - {doc}")

    # مثال ۲: مقایسه مدل بولی و مدل رتبه‌بندی‌شده
    print("\n" + "="*60)
    print("مثال ۲: مقایسه مدل بولی و مدل رتبه‌بندی‌شده")
    query_terms = ["میزبان", "مسئولیت"]

    # نتایج مدل بولی
    boolean_results = boolean_search(documents, query_terms)
    print(f"نتایج مدل بولی (بدون رتبه‌بندی):")
    for doc in boolean_results:
        print(f" - {doc}")

    # نتایج مدل رتبه‌بندی‌شده
    ranked_results = ranked_retrieval(documents, query_terms)
    print(f"نتایج مدل رتبه‌بندی‌شده (با امتیاز):")
```

```
for doc, score in ranked_results:
    print(f"    امتیاز {score}: {doc}")
```

```
# مثال ۳: نمایش فراوانی واژه در اسناد مختلف
print("\n" + "="*60)
print("مثال ۳: فراوانی واژه در اسناد مختلف")
term = "میزبان"
print(f"\nدر اسناد '{term}' فراوانی واژه:")
for doc in documents:
    freq = term_frequency(doc, term)
    print(f"    {doc}: {freq} بار")
```

مثال ۱: مقایسه جستجوی بولی ساده و جستجوی نزدیکی

:(مهمان AND میزبان) نتایج جستجوی بولی ساده

- ...سند 1: مهمانان در مهمانی مست بودند. میزبان مسئولیت مواد ا -
- ...سند 2: میزبان مهمانی دیگری برگزار کرد. در این مهمانی، مهم -
- ...سند 3: قوانین مربوط به مسئولیت میزبان در بسیاری از کشورها -
- ...سند 4: مهمان و میزبان توافق‌نامه‌ای قبل از مهمانی امضا کر -
- ...سند 5: در این مهمانی، میزبان به مهمانان الکل نداد. با این -

:(مهمان p / میزبان) نتایج جستجوی نزدیکی

- ...سند 1: مهمانان در مهمانی مست بودند. میزبان مسئولیت مواد ا -
- ...سند 2: میزبان مهمانی دیگری برگزار کرد. در این مهمانی، مهم -
- ...سند 3: قوانین مربوط به مسئولیت میزبان در بسیاری از کشورها -
- ...سند 4: مهمان و میزبان توافق‌نامه‌ای قبل از مهمانی امضا کر -
- ...سند 5: در این مهمانی، میزبان به مهمانان الکل نداد. با این -

مثال ۲: مقایسه مدل بولی و مدل رتبه‌بندی‌شده

:نتایج مدل بولی (بدون رتبه‌بندی)

- ...سند 1: مهمانان در مهمانی مست بودند. میزبان مسئولیت مواد ا -
- ...سند 2: میزبان مهمانی دیگری برگزار کرد. در این مهمانی، مهم -
- ...سند 3: قوانین مربوط به مسئولیت میزبان در بسیاری از کشورها -
- ...سند 4: مهمان و میزبان توافق‌نامه‌ای قبل از مهمانی امضا کر -

:نتایج مدل رتبه‌بندی‌شده (با امتیاز)

- ...امتیاز 3: سند 2: میزبان مهمانی دیگری برگزار کرد. در این مهمانی، مهم -
- ...امتیاز 3: سند 3: قوانین مربوط به مسئولیت میزبان در بسیاری از کشورها -
- ...امتیاز 2: سند 1: مهمانان در مهمانی مست بودند. میزبان مسئولیت مواد ا -
- ...امتیاز 2: سند 4: مهمان و میزبان توافق‌نامه‌ای قبل از مهمانی امضا کر -
- ...امتیاز 1: سند 5: در این مهمانی، میزبان به مهمانان الکل نداد. با این -

مثال ۳: فراوانی واژه در اسناد مختلف

:فراوانی واژه 'میزبان' در اسناد

- سند 1: مهمانان در مهمانی مست بودند. میزبان مسئولیت مواد ا...: 1 بار -
- سند 2: میزبان مهمانی دیگری برگزار کرد. در این مهمانی، مهم...: 2 بار -
- سند 3: قوانین مربوط به مسئولیت میزبان در بسیاری از کشورها...: 2 بار -
- سند 4: مهمان و میزبان توافق‌نامه‌ای قبل از مهمانی امضا کر...: 1 بار -
- سند 5: در این مهمانی، میزبان به مهمانان الکل نداد. با این...: 1 بار -

جمع‌بندی فصل ۱: بازیابی بولی

در این فصل با مفاهیم پایه‌ای بازیابی اطلاعات آشنا شدیم:

- مدل بولی را به‌عنوان سنگ بنای سیستم‌های جستجو معرفی کردیم. این مدل همان منطقی است که در کتابخانه‌ها با جستجوی «و» و «یا» استفاده می‌کنیم - مثلاً جستجوی «هوش مصنوعی و یادگیری ماشین» در پایگاه داده مقالات علمی.
- ماتریس وقوع بولی و محدودیت‌های آن را بررسی کردیم. تصور کنید یک کتابخانه با هزاران کتاب دارید و می‌خواهید بدانید کدام کتاب‌ها کلمات کلیدی خاصی دارند - ماتریس وقوع این کار را انجام می‌دهد، اما برای کتابخانه‌های بزرگ بسیار ناکارآمد است!
- فهرست معکوس را به‌عنوان راه‌حلی کارآمد برای ذخیره‌سازی و جست‌وجو معرفی کردیم. این همان فهرستی است که در انتهای کتاب‌ها می‌بینیم، اما برای جستجوی کامپیوتری بهینه شده است. گوگل در نسخه‌های اولیه خود از این روش استفاده می‌کرد.
- الگوریتم‌های اشتراک (AND)، ادغام (OR) و بهینه‌سازی ترتیب پردازش را پیاده‌سازی کردیم. وقتی در اسنپ فود «پیتزا و نوشابه» را جستجو می‌کنید، در واقع از الگوریتم AND استفاده می‌کنید - فقط نتایجی نمایش داده می‌شوند که هر دو کلمه را داشته باشند.
- تفاوت مدل بولی ساده، بولی گسترش‌یافته، و بازیابی رتبه‌بندی‌شده را بررسی کردیم. این همان تفاوتی است که بین جستجوی ساده در فایل‌های ویندوز و جستجوی پیشرفته در گوگل وجود دارد.

بیاید این مفاهیم را با چند مثال واقعی مقایسه کنیم:

♦ **بازیابی بولی ساده:** تصور کنید در دیجی‌کالا به دنبال «گوشی سامسونگ با حافظه ۲۵۶ گیگابایت و قیمت زیر ۱۰ میلیون تومان» می‌گردید. این جستجو دقیقاً از منطق بولی استفاده می‌کند: (گوشی AND سامسونگ) AND (حافظه ۲۵۶) AND (قیمت زیر ۱۰ میلیون). سیستم ابتدا تمام گوشی‌های سامسونگ را پیدا می‌کند، سپس از میان آن‌ها موارد با حافظه ۲۵۶ را جدا کرده و در نهایت آن‌هایی را نمایش می‌دهد که قیمت مناسب داشته باشند. نتیجه یا هست یا نیست؛ هیچ رتبه‌بندی وجود ندارد.

♦ **بازیابی بولی گسترش‌یافته:** یک وکیل در پایگاه داده قوانین و آراء دادگاه به دنبال یک مورد خاص می‌گردد. او جستجوی زیر را انجام می‌دهد:

«مسئولیت قراردادی NEAR/10 خسارت»

در اینجا، `NEAR/10` یک عملگر گسترش‌یافته است که به سیستم می‌گوید فقط سندهایی را پیدا کن که عبارت «مسئولیت قراردادی» و «خسارت» در فاصله کمتر از ۱۰ کلمه از هم قرار دارند. این کار از پیدا کردن سندهایی که این دو کلمه را به صورت پراکنده در آن آورده‌اند، بسیار دقیق‌تر است.

♦ **بازیابی رتبه‌بندی‌شده:** شما در گوگل عبارت «بهترین کافه‌های قصرالدشت» را جستجو می‌کنید. گوگل هزاران صفحه را پیدا می‌کند که این کلمات را دارند. اما به جای نمایش همه آن‌ها، آن‌ها را **رتبه‌بندی** می‌کند. نتیجه اول احتمالاً کافه‌ای است که امتیاز بالایی دارد، نظرات مثبت زیادی گرفته، به موقعیت مکانی شما نزدیک است و در وبسایت‌های معتبر به آن اشاره شده است. نتیجه پنجم ممکن است کافه خوبی باشد اما امتیاز کمتری دارد. این مدل، نتایج را بر اساس «مرتبطی» به شما نمایش می‌دهد، نه فقط بر اساس تطابق کلمات.

این فصل پایه‌های سخت‌افزاری و ساختاری سیستم‌های جست‌وجو را فراهم می‌کند. در فصل‌های بعدی، با مفاهیمی مانند:

- وزن‌دهی به واژه‌ها (tf-idf) - همان روشی که گوگل تشخیص می‌دهد کلمه «گوشی» در یک صفحه مهم‌تر از کلمه «و» است
- مدل فضای برداری - تکنیکی که به سیستم‌های جست‌وجو اجازه می‌دهد مفاهیم مشابه را پیدا کنند، حتی اگر کلمات یکسانی استفاده نشده باشد
- ارزیابی سیستم‌ها (دقت، بازیابی، F1) - معیارهایی برای سنجش اینکه آیا موتور جست‌جوی ما واقعاً خوب کار می‌کند یا نه
- خوشه‌بندی و طبقه‌بندی متن - روشی که گوگل اخبار را به دسته‌های ورزشی، سیاسی و اقتصادی تقسیم می‌کند

آشنا خواهیم شد – که همگی بر همین پایه فهرست معکوس بنا شده‌اند.